

Empirical Analysis and Mitigation of Vanishing Gradients in Deep Neural Networks

Rajesh Phulwaria
Independent Researcher
Email: phulwariarajesh152@gmail.com

Abstract—Deep neural networks often suffer from the vanishing gradient problem, where gradients diminish as they propagate backward through layers, limiting effective learning. In this study, we experimentally analyze gradient flow in a deep sigmoid-based neural network and compare it with a stabilized architecture using ReLU activation, improved initialization, and optimized loss functions. Through empirical evaluation, including gradient norm visualization and loss curves, we demonstrate how architectural choices directly impact training stability and performance.

I. INTRODUCTION

Deep learning models rely on backpropagation to update parameters. However, in deep architectures, gradients may shrink exponentially, leading to ineffective learning in earlier layers.

This problem is known as the **vanishing gradient problem**.

Objective of this study:

- To experimentally observe vanishing gradients
- To quantify gradient decay across layers
- To compare with a stable neural network design

Unlike purely theoretical analysis, this work focuses on **practical experimentation and visualization**.

II. METHODOLOGY

A. Model Architectures

Two neural networks were implemented:

1) Vanishing Gradient Model:

- Activation: Sigmoid (all layers)
- Output: Sigmoid
- Loss: BCELoss
- Optimizer: SGD (lr = 0.01)

2) Stable Model:

- Activation: ReLU
- Output: Linear
- Loss: BCEWithLogitsLoss
- Optimizer: Adam (lr = 0.001)
- Initialization: He Initialization

B. Architecture

Input $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 8 $\rightarrow$ 1

C. Gradient Measurement

Gradient norms were computed as:

$$\|g\| = \sqrt{\sum g_i^2} \quad (1)$$

This provides a scalar measure of gradient strength in each layer.

III. RESULTS

A. Gradient Flow Analysis

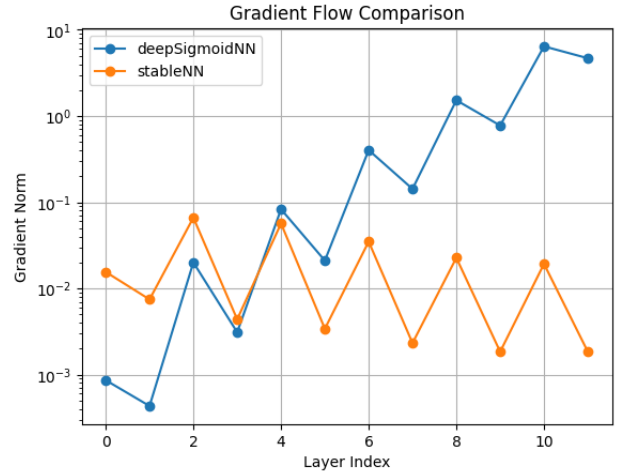


Fig. 1. Gradient Flow Comparison Across Layers

Observations:

- Sigmoid model shows **exponential increase** in gradient magnitude from early to later layers
- Early layers have extremely small gradients ($\sim 10^{-6}$)
- Stable model maintains **consistent gradient magnitude** across layers

Interpretation:

This confirms that in sigmoid networks:

- Gradients vanish as they propagate backward
- Early layers receive negligible updates

In contrast, ReLU preserves gradient flow, enabling all layers to learn effectively.

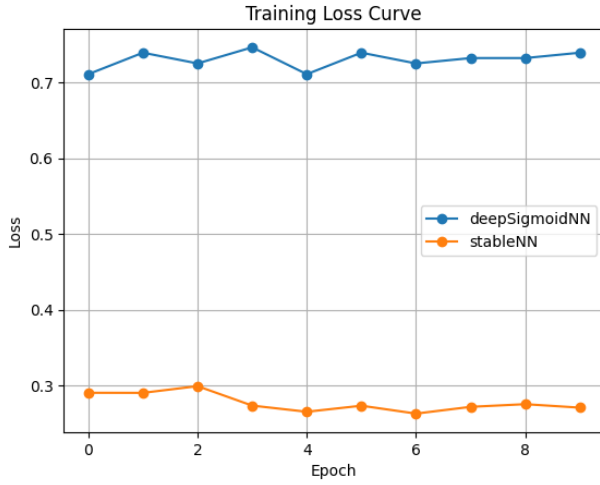


Fig. 2. Training Loss Comparison

B. Training Loss Analysis

Observations:

- Sigmoid model shows **unstable and oscillating loss**
- Stable model shows **smooth and consistent convergence**

Interpretation:

- Vanishing gradients lead to weak and inconsistent learning signals
- Stable gradients enable efficient optimization

IV. DISCUSSION

A. Why Vanishing Gradients Occur

Sigmoid derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \quad (2)$$

Maximum value:

$$\sigma'(x) \leq 0.25 \quad (3)$$

Thus, during backpropagation:

$$g \propto (0.25)^L \quad (4)$$

This leads to exponential decay of gradients with depth.

B. Why the Stable Model Works

- ReLU derivative is 1 for positive inputs
- He initialization maintains variance
- BCEWithLogitsLoss improves numerical stability

V. DEPENDENCIES AND IMPLEMENTATION DETAILS

A. Libraries Used

- **PyTorch**: Core deep learning framework used for model building and training
- **NumPy**: Used for numerical operations and gradient aggregation
- **Matplotlib**: Used for plotting gradient flow and loss curves

B. Why These Matter

- PyTorch provides automatic differentiation (autograd), enabling gradient tracking
- NumPy allows efficient numerical computation for analysis
- Matplotlib enables visualization, which is crucial for understanding model behavior

Reproducibility: All experiments are implemented in Python, ensuring reproducibility and extensibility.

VI. KEY INSIGHTS

- Vanishing gradients do not completely stop learning but severely limit it
- Deep sigmoid networks behave like shallow networks due to frozen early layers
- Proper architectural choices eliminate this issue

VII. CONCLUSION

This study empirically demonstrates that:

- Sigmoid-based deep networks suffer from vanishing gradients
- Gradient decay is exponential with depth
- Stable architectures maintain consistent gradient flow

In conclusion, this work demonstrates that the effectiveness of deep neural networks is governed not by depth alone, but by the stability of gradient flow, which is critically determined by activation functions, initialization schemes, and loss design.